

Step 2 - Propose High-Level Design and Get Buy-In

Instead of directly presenting a complex distributed architecture, we begin with a very simple design and gradually scale it to support millions of users. This approach helps explain the evolution of the system clearly and highlights the reasoning behind each architectural improvement.

Initial Single Server Design

At the beginning, the entire cloud storage system can run on a single server setup consisting of:

- A web server for handling file uploads and downloads
- A relational database for storing metadata
- A local storage system for storing actual files

The setup contains:

- **Apache/Nginx Web Server**
- **MySQL Database**
- **1 TB local disk storage**

This simple architecture is sufficient for early-stage development and small-scale traffic.

Components of the Initial System

Web Server

The web server handles:

- File uploads
- File downloads
- User authentication
- File revision APIs

It exposes REST APIs over HTTPS.

Database

The database stores metadata such as:

- User profile information
- Login credentials
- File metadata
- File ownership
- Sharing permissions
- Revision history

The database does **not** store the actual file content.

Storage System

Actual files are stored on disk under a root directory called:

```
None  
/drive
```

Each user gets a separate namespace directory.

Example structure:

```
None  
/drive  
  
  /user_A  
  
    photo.png  
  
    docs/report.pdf
```

```
/user_B
```

```
notes.txt
```

Each file can be uniquely identified using:

Namespace + Relative Path

For example:

None

```
/user_A/docs/report.pdf
```

Single Server File Storage Structure



Figure illustrates the `/drive` directory organization:

- The left side shows the root `/drive` directory.
- The right side shows expanded user namespaces containing uploaded files and folders.

This namespace-based structure helps isolate user files and simplifies file management.

Core APIs

The system primarily requires three APIs:

1. Upload file
2. Download file
3. Get file revisions

1. Upload File API

The upload API supports two upload modes.

Simple Upload

Used when:

- File size is small
- Network interruption probability is low

Resumable Upload

Used when:

- File size is large
- Upload may be interrupted

Example API:

None

```
https://api.example.com/files/upload?uploadType=resumable
```

Parameters

JSON

```
{  
  "uploadType": "resumable",  
  "data": "local_file"  
}
```

Resumable Upload Flow

A resumable upload works in three steps:

Step 1 — Request Upload URL

The client sends an initial request to obtain a resumable upload URL.

Step 2 — Upload File Chunks

The client uploads file chunks while monitoring upload progress.

Step 3 — Resume on Failure

If the network connection fails, the upload resumes from the last successful chunk instead of restarting from the beginning.

This approach:

- Improves reliability
- Reduces bandwidth waste
- Enhances user experience

2. Download File API

Example API:

```
None  
https://api.example.com/files/download
```

Parameters

```
JSON  
  
{  
  
  "path": "/recipes/soup/best_soup.txt"  
  
}
```

The server locates the file using the namespace and relative path, then streams the file back to the client.

3. Get File Revisions API

This API retrieves file version history.

Example API:

```
None  
https://api.example.com/files/list\_revisions
```

Parameters

```
JSON  
  
{  
  
  "path": "/recipes/soup/best_soup.txt",
```

```
"limit": 20
```

```
}
```

File Revision Support

File revisions allow users to:

- Restore previous versions
- Recover deleted content
- Track modification history

Revision metadata is stored in the database.

Security Considerations

All APIs require:

- User authentication
- Authorization checks
- HTTPS communication

HTTPS uses SSL/TLS encryption to secure data transfer between clients and servers.

Limitations of Single Server Design

Although the single server setup is simple, it has several limitations:

Limited Storage Capacity

A single disk cannot scale to petabytes of data.

Single Point of Failure

If the server crashes:

- File uploads fail
- Downloads stop working
- Metadata becomes inaccessible

Poor Scalability

The architecture cannot efficiently support:

- Millions of concurrent users
- High upload/download traffic
- Global users

Slow Synchronization

Syncing files across multiple devices becomes inefficient at scale.

Need for Distributed Architecture

As the system grows, we must introduce:

- Distributed file storage
- Load balancers
- Metadata sharding
- Replication
- File synchronization services
- Notification systems
- Chunk-based storage
- CDN support

These improvements will be discussed in the deep dive section.